

PROYECTO DE INFRAESTRUCTURA

NUBE PERSONAL

Servidor Dedicado Low Cost

Nextcloud · Docker · Cloudflare Tunnel · Tailscale · MCP

 Seguro ·  Backup 3-2-1 ·  Monitoreo en 3 capas ·  Gestión con IA

Documento de presentación y estudio

Junio 2026 · Costo anual ~\$46 USD

Resumen ejecutivo

Nube Personal con Servidor Dedicado Low Cost Documento de presentación y estudio · Junio 2026

El proyecto en una frase

Una **nube privada propia** (estilo Google Drive / Dropbox) que corre en un mini-PC doméstico, accesible de forma segura desde cualquier lugar del mundo, con **control total de los datos** y un costo operativo de **~\$46 USD al año**.

Por qué

Motivación	Resultado
Privacidad y control de datos	Los archivos viven en hardware propio, sin terceros
Costo	~\$46/año vs \$120-144/año de servicios comerciales (2 TB)
Aprendizaje	Proyecto integral de SysAdmin, Redes, DevOps, Seguridad y Cloud
Capacidad	1.7 TB útiles, usuarios ilimitados, personalización total

Logros clave

- ✓ **Nube funcional 24/7** en <https://cloud.hexa38.com>
- ✓ **Acceso seguro global** sin abrir puertos (Cloudflare Tunnel + Tailscale)
- ✓ **Backup 3-2-1** automático (local 7 días + Google Drive 14 días)
- ✓ **Sincronización instantánea** (Notify Push vía WebSocket)
- ✓ **Monitoreo en 3 capas** (auto-recuperación + alertas Telegram/Email)
- ✓ **Gestión asistida por IA** mediante un servidor MCP propio (11 herramientas)
- ✓ **Diagnóstico forense multicapa** que resolvió un problema real de red, **verificado con datos** (–98.7 % de inestabilidad)

Cifras del proyecto

Métrica	Valor
Costo anual	~\$46 USD
Ahorro vs comerciales	60-68 %
Almacenamiento útil	1.7 TB
Contenedores Docker	6
Herramientas MCP de gestión	11
Capas de monitoreo	3
Retención de backup	7 días local + 14 días off-site
Tiempo de arranque	< 2 minutos

1. Arquitectura

1.1 Visión general



1.2 Hardware

Componente	Especificación	Función
NUC Intel N3150	4 GB RAM, WiFi/Ethernet	Servidor 24/7
Disco M.2 SSD	297 GB	Sistema (Ubuntu Server 24.04)
Disco WD externo	2 TB (1.8 TB útiles, ext4)	Datos de usuarios

1.3 Componentes de software (6 contenedores Docker)

Contenedor	Imagen	Función
<code>nextcloud-app</code>	<code>nextcloud:latest</code>	App web + Apache (reverse proxy)
<code>nextcloud-db</code>	<code>mariadb:10.11</code>	Base de datos
<code>nextcloud-redis</code>	<code>redis:alpine</code>	Caché + file locking
<code>nextcloud-cron</code>	<code>nextcloud:latest</code>	Tareas programadas
<code>nextcloud-tunnel</code>	<code>cloudflare/cloudflared</code>	Túnel a Internet
<code>nextcloud-push</code>	<code>nextcloud:latest</code>	Sincronización instantánea

1.4 Stack y disciplinas

Stack: Nextcloud · Docker / Docker Compose · MariaDB · Redis · Cloudflare Tunnel · Tailscale · rclone · MCP (Python).

Disciplinas aplicadas: SysAdmin, Redes, DevOps, Seguridad, Bases de datos, Scripting, Cloud Computing, Infrastructure as Code, Forense de sistemas.

2. Seguridad

Medida	Estado
HTTPS forzado + HSTS	✓
2FA / TOTP obligatorio	✓
Firewall UFW (solo SSH y Nextcloud)	✓
Cloudflare como WAF	✓
Sin puertos abiertos al exterior (túnel)	✓
SSH remoto cifrado por Tailscale (WireGuard)	✓
Directorio <code>/data</code> protegido	✓
Updates automáticos de seguridad	✓

Filosofía: *defense in depth* — varias capas independientes, de modo que ninguna falla aislada compromete el sistema.

3. Resiliencia: backup y monitoreo

3.1 Backup (regla 3-2-1)

- **3** copias · **2** medios · **1** off-site
- Backup local diario a las 3 AM (retención 7 días)
- Backup off-site en Google Drive vía rclone (retención 14 días)
- El script activa modo mantenimiento, exporta la BD, copia datos y configuración, y limpia backups antiguos.

3.2 Monitoreo en 3 capas

Capa	Qué hace	Tiempo de reacción
1 — Docker healthchecks	Reinicia contenedores colgados (auto-recuperación)	Inmediato
2 — Healthchecks.io	Avisa si Nextcloud no responde (Telegram + Email)	10-15 min
3 — Cloudflare Tunnel alerts	Avisa si el túnel/NUC pierde conectividad (Email)	1-2 min

4. Gestión asistida por IA (servidor MCP)

4.1 Qué es

Un **servidor MCP (Model Context Protocol)** propio que permite a Claude (IA) **diagnosticar y operar el NUC directamente desde una conversación**, sin abrir SSH manualmente. El monitoreo *avisa* del problema; el MCP permite *resolverlo* de forma asistida.

4.2 Herramientas (11)

`check_health` · `get_containers` · `get_logs` · `restart_service` · `run_occ` · `check_backup` · `check_disk` · `check_tunnel` · `maintenance_mode` · `run_command` · `debug_connection` .

4.3 Decisión técnica destacada

Claude Desktop ejecuta el MCP en un **sandbox que bloquea** `ssh.exe` como proceso hijo. La solución fue usar **paramiko** (SSH dentro del propio proceso Python) con autenticación por **identidad de red de Tailscale** (`auth_none` , sin claves ni contraseñas). Resultado: un canal de gestión seguro y sin secretos en disco.

5. Caso de estudio: la alerta que venía de tres capas más abajo

Un ejemplo real del valor del proyecto y su metodología.

5.1 El síntoma

Cloudflare enviaba correos recurrentes de **"túnel degradado"**.

5.2 El diagnóstico (de lo general a lo específico)

1. **Logs del túnel** → las 4 conexiones QUIC caían simultáneamente ("no recent network activity").
2. **Interfaces de red** → el NUC estaba en **WiFi**; el Ethernet (`enp3s0`) estaba caído.
3. **Logs del kernel** → tormenta de "Lost carrier" en el WiFi, coincidiendo en *timestamp* con las caídas del túnel.
4. **Hardware** → chip **Intel 3160 (driver `iwlwifi`)** con señal fuerte (-49 dBm) pero cortes → **bug de gestión de energía del driver**, no señal débil.

5.3 La causa raíz

El ahorro de energía agresivo del driver `iwlwifi` apagaba la radio momentáneamente, matando las conexiones del túnel. **Las alertas eran reales**; el túnel estaba bien configurado.

5.4 La solución

Desactivar el power management del WiFi de forma persistente (`/etc/modprobe.d/iwlwifi.conf` : `power_save=0` , etc.).

5.5 La evidencia (verificada con MCP, monitoreo de 24 h)

Métrica	ANTES	DESPUÉS	Mejora
Eventos de carrier WiFi / hora	52	~0.7	-98.7 %
Horas con cero eventos	0 de 1	16 de 17	—
Desconexiones del túnel	continuas	0 en 18.5 h	 resuelto

Lección: una alerta correcta puede tener su causa raíz a varias capas de distancia (aplicación → red → kernel → hardware). Y no basta con aplicar el fix: **medir** es lo que convierte "creo que lo arreglé" en "está resuelto, aquí están los números".

6. Conclusiones

- El proyecto logra una **nube personal segura, resiliente y de bajo costo**, operativa 24/7.
 - Combina **buenas prácticas de infraestructura** (IaC, contenedores, backup 3-2-1, monitoreo en capas) con **innovación** (gestión asistida por IA vía MCP).
 - El caso de la inestabilidad WiFi demuestra una **metodología de diagnóstico rigurosa y verificable con datos**.
 - Próxima mejora opcional: **cable Ethernet** para robustez extra; reemplazo del HDD de sistema por SSD a mediano plazo.
-

Anexo A — Configuración técnica clave

A.1 docker-compose (extracto)

```
services:
  db:
    image: mariadb:10.11
    command: --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-format=ROW
    volumes: [ /mnt/nextcloud/db:/var/lib/mysql ]
  redis:
    image: redis:alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
  app:
    image: nextcloud:latest
    ports: [ "9000:80" ]
    depends_on: [ db, redis ]
    volumes:
      - /mnt/nextcloud/html:/var/www/html
      - /mnt/nextcloud/data:/var/www/html/data
    environment:
      - MYSQL_HOST=db
      - REDIS_HOST=redis
      - OVERWRITE_PROTOCOL=https
```

A.2 Healthchecks Docker (ejemplo: app)

```
healthcheck:
  test: ["CMD-SHELL", "php -r 'exit(@file_get_contents(\"http://localhost/status.php\") ? 0 : 1);'"]
  interval: 60s
  timeout: 10s
  retries: 3
  start_period: 120s
```

A.3 Fix de estabilidad WiFi (Intel 3160 / iwlwifi)

```
sudo tee /etc/modprobe.d/iwlwifi.conf > /dev/null << 'EOF'
options iwlwifi power_save=0 d0i3_disable=1 uapsd_disable=1
options iwlvm power_scheme=1
EOF
sudo modprobe -r iwlvm iwlwifi && sudo modprobe iwlwifi
iw dev wlp2s0 get power_save      # → Power save: off
```

Anexo B — Servidor MCP (núcleo)

```
# SSH in-process con paramiko (compatible con el sandbox de Claude Desktop)
# Tailscale SSH autentica por identidad de red (auth_none): sin clave ni password
import socket, paramiko

sock = socket.create_connection((NUC_HOST, 22), timeout=15)
transport = paramiko.Transport(sock)
transport.start_client(timeout=15)
transport.auth_none(NUC_USER)          # Tailscale SSH
chan = transport.open_session()
chan.exec_command(command)
```

Registro en Claude Desktop (`claude_desktop_config.json`):

```
{ "mcpServers": { "nextcloud-nuc": {
  "command": "python.exe",
  "args": [ "...\\mcp-nextcloud\\server.py" ] } } }
```

Anexo C — Comandos de operación frecuentes

```
# Diagnóstico rápido
uptime; docker ps; df -h; free -h
systemctl --failed
sudo journalctl --since "1 hour ago" -p err --no-pager | head -30

# Nextcloud (occ)
docker exec -u www-data nextcloud-app php occ status
docker exec -u www-data nextcloud-app php occ user:list
docker exec -u www-data nextcloud-app php occ maintenance:mode --off

# Backup manual y verificación off-site
sudo bash /home/aldo/nextcloud/backup.sh
rclone ls gdrive:nextcloud-backups/

# Red / WiFi
ip -brief addr show
journalctl --since '1 day ago' | grep -iE 'carrier|dhcp lease'

# Túnel
docker logs nextcloud-tunnel --tail 80
```

Anexo D — Glosario esencial

Término	Definición breve
Cloudflare Tunnel	Conexión cifrada a Internet sin abrir puertos ni IP pública
Tailscale	VPN mesh (WireGuard) para acceso remoto seguro
Notify Push	Servidor de sincronización instantánea vía WebSocket
Redis / APCu / OPcache	Capas de caché para rendimiento
Regla 3-2-1	3 copias, 2 medios, 1 off-site
MCP	Model Context Protocol: permite a una IA usar herramientas externas
iwlwifi	Driver del chip WiFi Intel; su power management causó la inestabilidad
SMART	Tecnología de autodiagnóstico de discos

*Documento generado para presentación y estudio del proyecto. Para el detalle exhaustivo (procedimientos paso a paso, recuperación ante desastres, catálogo completo de comandos), ver la **Guía Definitiva — Nextcloud Hexa38 (v3.2)**.*